

OOP:-

- * class → ctor, method, overloading
- * Inheritance → virtual, override, new keyword
- * Interface → Syntax, template, public contract, changing implementation of methods, filtration
- * Abstract → Application Execution flow we can decide by writing base abstract entities. [Sequential Execution]
template, force full implementation, common functionality extract.
- Static → Logger → Private ctor, Singleton, Object Pooling

- Factory Pattern,
- Constructor level Dependency Injection
Pattern → interface / abstract

- Singleton

- SRP

- OCP

- DRY

- YAGNI

✓, ✓, ✓, ✓, ✓
S, O, L, I, D

✓ Access modifiers :- public, private,
protected, internal,
protected-internal

✓ Properties

✓ Sealed classes / methods.

✓ Class Library :- dll

✓ Events & Delegates. ✓

Console → C# → ☒ Do not use top-level statement.

Library → C#

old.NET :- Form Application → Desktop based app.
↓ calculator app.

Extensible Markup language [XAML]
↳ Windows Presentation Foundation [WPF]

WPF → XAML → XML + markup

e.g. - Visual Studio

Windows Communication Foundation

[WCF] $\rightarrow$ BSE $\rightarrow$ o/p..

$\hookrightarrow$.Net web services.

Win32UI ✓, MAUI ✓ ... etc.

Blaze ✓

Xamine

A Delegate is a type-safe
function pointer.

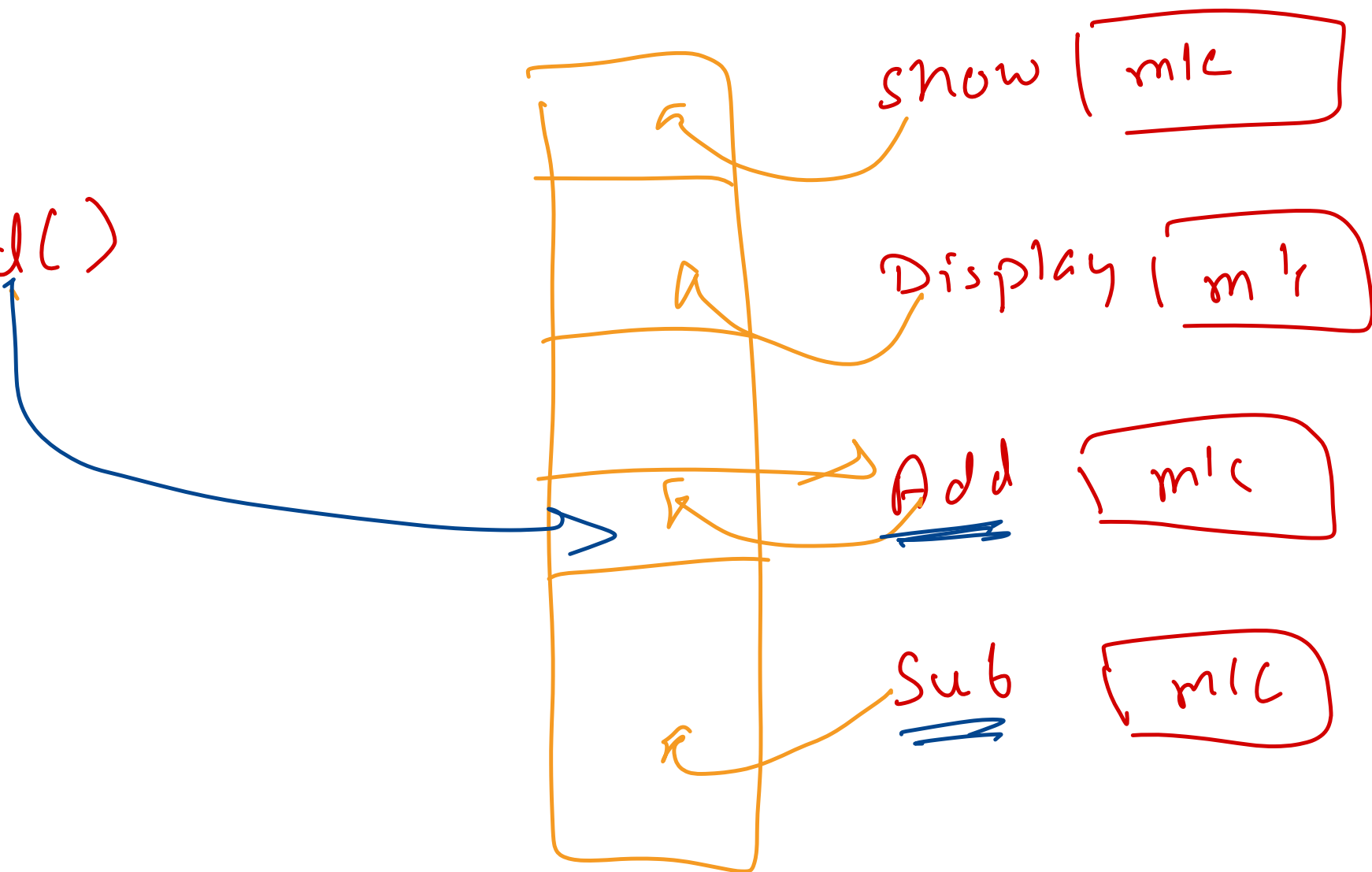
delegate void () → void display() { }
void show() { }

delegate void (string nm)

void Greet (string nm) { }

delegate int (int x, int y) :- Add, Sub,
mult, Div.

Add()



function Registry.

public delegate void EventHandler ✓

(object sender ,
EventArgs e)

public void btn_click (object sender ,
EventArgs e)

?

✓